

Eccezioni in Python

Introduzione

In Python, gli **errori** che si verificano durante l'esecuzione del programma sono chiamati **eccezioni**. Se non gestite, possono causare l'interruzione del programma.

Per gestire gli errori si usa il blocco `try - except`.

Struttura base del `try - except`

La struttura per usare questo particolare blocco è:

1. Nel blocco `try` :

ci si mette il codice che può generare un'eccezione.

2. mentre nell' `except` :

si mette il codice eseguito se si verifica l'eccezione.



Python

```
1  try:
2      # Codice che potrebbe generare un'eccezione
3  except NomeEccezione:
4      # Codice eseguito se si verifica l'eccezione
5
```

Esempio:



Python

```
1  try:
2      linux_interaction()
3  except RuntimeError as error:
4      print(error)
```

 **Il `try` ed `except` è un blocco di codice all'interno del nostro programma e viene scritto quando si è consapevoli che quel particolare tipo di blocco di codice causerà degli errori o eccezioni**

Keyword aggiuntive

A parte il `try` e l' `except` , in Python vi sono altre keyword.

- `else` :

in questo blocco il codice viene eseguito solo se non si verifica nessuna eccezione.

Esempio:



Python

```
1  try:
2      linux_interaction()
3  except RuntimeError as error:
4      print(error)
5  else:
6      print("Doing even more Linux things")
```

2. `finally` : **viene eseguito sempre, sia se si verifica un'eccezione che no. Utile per chiudere file, liberare risorse, ripristinare variabili.**

Esempio:



Python

```
1  try:
2      linux_interaction()
3  except RuntimeError as error:
4      print(error)
5  else:
6      try:
7          with open("file.log") as file:
8              read_data = file.read()
9          except FileNotFoundError as fnf_error:
10             print(fnf_error)
11  finally:
12      print("Cleaning up, irrespective of any exceptions")
```

BaseException e gerarchia delle eccezioni

In Python esistono molte eccezioni([Qui per avere la lista completa](#)), quelle più usate sono

1. BaseException :

è la classe madre di tutte le eccezioni di Python.

2. `except Exception:`

è una sottoclasse di `BaseException` e serve per usare eccezioni generiche come `ValueError`, `TypeError`, `FileNotFoundError`, etc.

🔗 **Differenza tra** `Basic Exception` **e** `except Exception` >

1. `Basic Exception`

- È la vera classe madre di *tutte* le eccezioni in Python.
- È superclasse anche di eccezioni speciali come:
 - `SystemExit` (quando il programma sta per chiudersi)
 - `KeyboardInterrupt` (quando premi `Ctrl+C`)
 - `GeneratorExit`

2. `Exception`

- È una sottoclasse di `BaseException`.
- È la classe madre della maggior parte delle eccezioni “normali”, come `ValueError`, `TypeError`, `FileNotFoundError`, ecc.
- Non include eccezioni “di sistema” come `KeyboardInterrupt`.

Perché allora si usa `except Exception` ?

Perché è più sicuro:



Python

```
1  except Exception:
2
```

Ciò significa:

Gestisco solo gli errori 'normali' del programma, ma lascio passare eccezioni critiche come un `Ctrl+C` o l'uscita del programma.

Mentre se uso `except BaseException:`



Python

```
1  except BaseException:
```

Ciò significa:

Catturo qualsiasi cosa, anche un'interruzione da tastiera o un'uscita del sistema.

Ovviamente è molto rischioso perché:

- Si impedisce la chiusura ordinata del programma
- Blocca comandi da tastiera rendendo difficile interrompere manualmente lo script.

Di conseguenza `BaseException` è utile solo in **casi rari**(es. logging avanzato, cleanup critico, debugger, ecc)

Creazione di eccezioni personalizzate

Python dà la possibilità di creare eccezioni personalizzate: supponiamo di star scrivendo un programma e vogliamo creare delle eccezioni custom.

Per fare ciò bisogna creare una classe che eredita da `Exception`.

Questo passaggio è molto importante perché: se non si eredita da `Exception`, Python vedrà quella classe come una classe qualunque, mentre se si eredita da `Exception` quella classe diventa una classe di eccezione.



Python

```
1 class PlatformException(Exception):  
2     "Incompatible platform"
```

La keyword `raise`

Finora abbiamo visto come in Python se si verificano errori viene sollevata un'eccezione ma può essere che in un nostro contesto quello che non è un errore per altri, per noi lo è e quindi torna utile sollevare un errore e per fare ciò si usa la keyword `raise`:

Serve a sollevare un'eccezione manualmente.

È utile quando una condizione che *non è tecnicamente un errore* lo diventa nel tuo contesto specifico.



Python

```
1 number=10  
2 if number >5:
```

```
3     raise Exception (f"The number should not exceedd 5.  
    ({number=})")  
4     print(number)
```

Questo ti blocca il programma.